

1. Installation guide

Pip + venv

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
copy .env.example .env
flask --app src.app run
```

uv

```
pip install uv
uv venv
.venv\Scripts\activate
uv pip sync requirements.txt
uv pip install -e .
copy .env.example .env
flask --app src.app run
```

2. Setup.py

Setup.py is important because it makes the project installable as a proper python package rather than a bunch of scripts. Packages are important because it allows you to do an editable install and they make the project portable

3. Dependency Graph

The graph shows app.py at the bottom as the root entry point, with arrows flowing upward to its dependencies. The left cluster is the Flask framework internals: flask.app is the core application object, which depends on flask.ctx (request/application context management), flask.globals (thread-local proxies like current_app and g), flask.wrappers (request and response objects), flask.helpers, flask.json, flask.blueprints, flask.templating, and flask.config. At the top, flask.signals are there and are used throughout Flask. The right cluster shows the database layer: psycopg2 is the PostgreSQL adapter, with psycopg2.extensions providing type adapters and psycopg2._psycopg being the compiled C extension that does the actual work. The psycopg2.sql module is specifically imported for safe SQL composition using sql.SQL () and sql.Identifier (). Finally, src.load_data and

src. query_data appear as internal project modules that app.py imports directly to handle data loading and analysis queries.

4. SQL injection

Every query in app.py and query_data is now wrapped in the sql.SQL() format from the spycopg2.sql module. Every query also has a explicit limit clause which clamps every function clamps any input between 1 and 100.

5. Least-Privilege DB configuration

The application uses a dedicated restricted account and configures a least privilege user. This is what the sql looks like

```
CREATE USER gradcafe_app WITH PASSWORD 'your_password';
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO gradcafe_app;
```

Connect allows the user to open a connection to the database and usage on schema public allows the user to see that the schema exists and can access the objects within it. The user is not granted the insert, update, delete, drop, alter, create privileges as they are not a superuser.

6. Snyk Dependency

The command, snyk test, was run against requirements.txt and the output was as follows

```
Your account has been authenticated.
• (.venv) PS C:\Users\ritwi\Documents\Modern Python\jhu_software_concepts\Module5> snyk test

Testing C:\Users\ritwi\Documents\Modern Python\jhu_software_concepts\Module5...

Organization:    ritwik2510
Package manager: pip
Target file:     requirements.txt
Project name:    Module5
Open source:     no
Project path:    C:\Users\ritwi\Documents\Modern Python\jhu_software_concepts\Module5
Licenses:        enabled

✓ Tested 46 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

No packages required patching or removal.

The command `snyk code test` was attempted but it returned the following error, Snyk Code is not enabled (SNYK-CODE-0005). This error occurs when Snyk Code is not enabled for the current Organization. Snyk Code is not supported for your current organization: ritwik2510.

This is because the snyk code feature needs to be enabled at the organizational level in the Snyk web dashboard which is not enabled in the free version used in this project. If it did work, it would perform a static analysis of the python source files and flag issues such as hardcoded secrets, injection vulnerabilities, and insecure deserializations at the code level rather than the dependency level.

7. Github actions

The CI pipeline is defined in `.github/workflows/ci.yml` and runs on every push and pull request. It contains four jobs:

Lint — installs pylint and runs `pylint src/ --fail-under=10`. The workflow fails if the score drops below 10.00/10, enforcing that no warnings or errors are introduced.

Dependency graph — installs Graphviz and pydeps, generates `dependency.svg`, then verifies the file exists with `test -f dependency.svg`. The SVG is uploaded as a build artifact.

Snyk — installs the Snyk CLI via npm and runs `snyk test --file=requirements.txt`. The `|| true` flag allows the job to continue even if vulnerabilities are found, so it produces output without blocking the pipeline.

Pytest — spins up a PostgreSQL 15 service container, creates the applicants table schema, installs the project with `pip install -e .`, and runs `pytest tests/ -v --tb=short` excluding `test_db_insert.py` (which requires a pre-loaded dataset). The job fails if any test fails.